Technical Assignment: Agentic Bug Triage & Resolution

Role: Director of Engineering, AI
Time: 48 hours

Why This Matters

Engineering teams lose hours every week on bug triage — reading through reports, figuring out what's actually broken, finding the right code, and writing fixes for things that could be automated. As codebases grow, this gets worse. The promise of agentic AI is that a system can observe a bug, reason about the codebase, and act on it — not just file a ticket, but actually fix the problem. We want to see how you'd build that.

The Task

Build an agentic workflow that triages and solves bugs.

Triage: When a bug is opened as a GitHub issue, the agent should reproduce it, analyze the root cause, classify severity, and route it appropriately.

Solve: For each triaged bug, the agent should propose and implement a fix — ideally with minimal human intervention where safe to do so.

Getting Started

Fork an existing open-source app to work against — you need a real codebase with backend, frontend, and a database. You can use something like:

- Vikunja — a full-featured Go/Vue task management app

Or pick any open-source app you're comfortable with. The app isn't the point — the agent is.

Introduce **at least 2 bugs** into the app — one backend and one frontend — and demonstrate your agent handling both end to end.

Tools & API Access

We're providing an **Anthropic API key** with $50 loaded. Use Haiku for development and testing, and Sonnet/Opus only when you need the extra reasoning power. The budget is intentional — how you manage model costs is part of the exercise.

Beyond that:

- **GitHub** — issues for bug tracking, PRs for fixes
- **Slack** — optional; feel free to spin up a free workspace if you want to show notifications or human-in-the-loop communication
- If your solution needs any tool or service that doesn't offer a free tier, let us know and we'll figure it out.

Use whatever frameworks or orchestration tools you prefer.

What We'll Evaluate

Functionality — Does it work? Can we open a bug and watch the agent triage and solve it end to end?

Autonomy level — How far does the agent get without human intervention? Is the level of autonomy appropriate to the risk?

Human-in-the-loop — Does the process keep humans informed at critical decision points? Is there a clear escalation path for things the agent isn't confident about?

Measurement — How do you know this is working? Are you tracking resolution time, success rate, or other signals?

Context & memory management — How does the agent understand the codebase? Does solving bug #1 help it solve bug #2 faster or better?

Taste — Is the output actually useful, or just technically impressive? Would a developer on a real team benefit from this, or would they mute it after a day?

Submission

Share your repo and any running agents or workflows, along with:

A video walkthrough (5–10 minutes) demoing the agent handling the bugs and explaining your design decisions. Show it running, not just slides.

A short write-up (1–2 pages max) covering:

- **What you built now** — the architecture, your choices, and why you made them

- **What you'd build with more time** — if you had a full month, what changes? What gets added, replaced, or rethought? This is where we see your strategic thinking.

Questions

If something is genuinely ambiguous and blocking you from making progress, reach out. But if you're deciding between two reasonable approaches, just pick one and explain your choice. Bias toward action.

Good luck :)